CM52060 – Bayesian Data Science Coursework

Hierarchical Bayesian Modelling of Mechanical Component Degradation

Consultancy Project Report - Part B

Academic Year 2025–2026

Contents

1 Introduction

In this report, a Bayesian analysis on the degradation pattern of mechanical ‘ "K" components will be presented for an engineering customer. The customer produces machinery which contains a crucial component that experiences heavy wear and tear over time. Degradation can be monitored based on the integrity level ranging from 0 to 100%, and the primary issue for the customer would be determining when the component degrades below 30%.

The client data consists of 879 readings of the integrity values for 75 components over 45 days, along with five more physical attributes (X1-X5) of each component that remain fixed throughout. Components ranging from index 0 to 49 are relatively well monitored, with a median number of around 16 readings per component and even some having as many as 40 readings. However, the rest of the 25 components (index 50 to 74), being installed later, have just one to eight readings each, all before day 10.

Modelling difficulties can be summarized into two major aspects. Firstly, future integrity prediction for those components lacking data should be reliable, whereas conventional statistical techniques and machine learning methods are not capable of accomplishing that task. Secondly, quantification of the effect of each of the five physical parameters on degradation rate should take place to inform the decision-making process concerning maintenance. Hierarchical Bayesian analysis was chosen as the technique since the client failed to achieve satisfactory results when experimenting with alternative solutions. (3).

The two Bayesian hierarchical models studied here are as follows: a baseline model with individual exponential decay based on the component (Equation 1) and the improved model taking into account five features of the physical properties as common predictors of the exponential decay rate (Equation 2). Both of the models have been implemented using HMC with NUTS using numpyro. (4).

2 Data Handling

The two models have the same data representation and inference approach (HMC/NUTS), except for the inclusion of the covariate feature X. as illustrated in Figure 1.

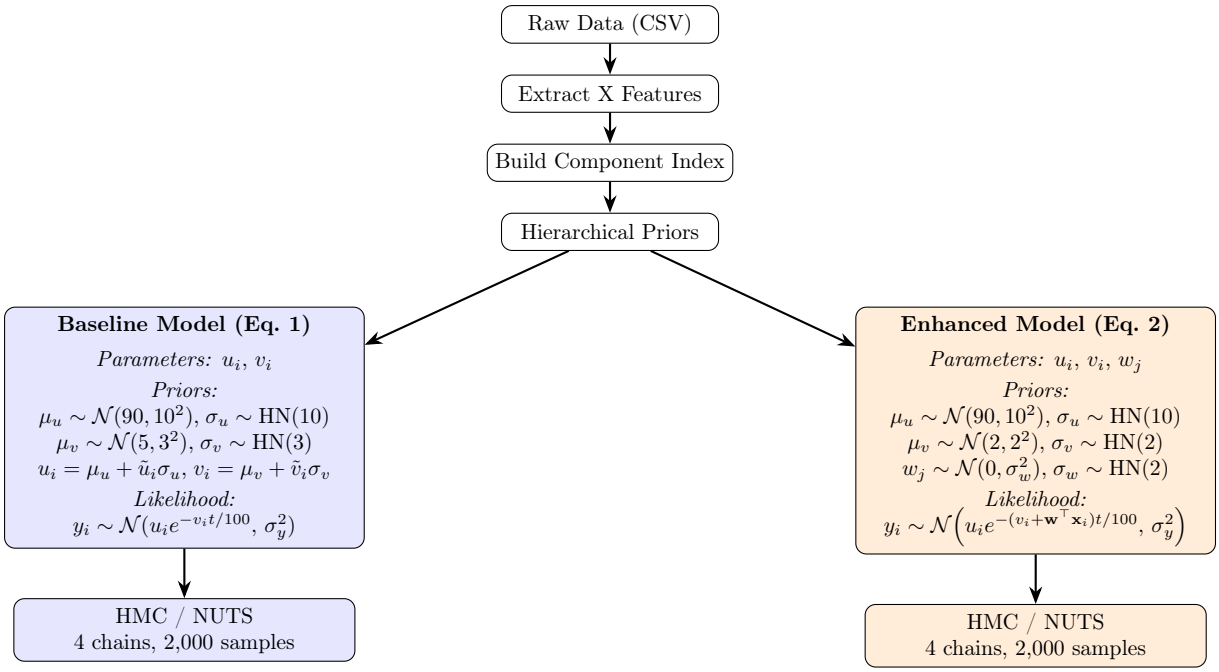

Figure 1: Both models share the same data preparation and inference algorithm, differing only in the inclusion of the X feature covariates. Each model box shows the estimated parameters, prior distributions, and observation likelihood. HN = HalfNormal.

2.1 Dataset Structure

The data table consists of the following information on each row: component ID, integer identifier (0-74), five real-number features X1-X5 (fixed over time for each component), day number $t \in [0, 45]$, and measurement of quality y $y \in [0, 100]$. The 75 components offer 879 data points in all: 805 data points are from the training components and 74 data points come from the testing components. There is sparse data in the testing dataset.

A diagnostic visualization of the data set (refer to, say, Fig. 4) confirms the hypothesis of exponential decay formulated earlier by the client. The magnitude of decay differs significantly between components, indicating significant differences between the components which must be accounted for using the hierarchy approach. Measurement noise is evident at the level of roughly $\pm 5\%$ integrity units, corresponding well to the calculated value of $\sigma_y = 5.1\%$.

2.2 Why Hierarchical Modelling?

The per-component strategy involves fitting u_i and v_i separately for each individual component based only on its data. This can be done for the 50 well-represented training components; although it is quite wasteful. It is impossible for the remaining 25 testing components that have just 1 to 4 measurements: with two parameters and only one measurement, the model becomes unidentifiable, leading to arbitrary predictions.

The hierarchical approach addresses the problem of using partial pooling (3). There is a common population distribution over u_i and v_i, which is learned based on the set of 50 well-sampled components simultaneously. This population distribution serves as a well-informed prior when fitting parameters for poorly sampled components. The amount of pooling is automatically determined by the data. Components with large amounts of data are mostly influenced by their own data, whereas those with small samples will be pulled toward the population mean proportionally to their sampling lack. In consequence, automatic regularization of predictions for poorly sampled components takes place, together with the appropriate increase of uncertainty

bands due to the lack of available data. This behavior is shown in Figure 4—the uncertainty bands for the sparsely sampled components are much larger compared to the well-sampled one. Most importantly, the prediction means for single observation components remain sensible thanks to the population distribution. A fully Bayesian approach also leads to a natural propagation of uncertainty throughout the model. Thus, the posterior predictive distribution over future integrity is a proper probability distribution, not just a single point estimation with an ad-hoc confidence band attached. This allows us to treat maintenance- relevant quantities like the probability of dropping below the 30% overhaul threshold as natural model outputs.

3 Modelling

3.1 Non-Centered Reparameterization

The two approaches both make use of a non-centered parameterization to enable more efficient HMC sampling (1). In the centered parameterization approach, u_i is sampled directly from $\mathcal{N}(\mu_u, \sigma_u^2)$. If one of the components contains few observations, the posterior distribution of u_i will be dominated primarily by its prior. The joint distribution takes on the form of a funnel-like structure in (μ_u, σ_u, u_i) space.

The non-centered parametrization circumvents this problem by drawing from a normalized deviation: $\tilde{u}_i \sim \mathcal{N}(0, 1)$, followed by applying a transformation $u_i = \mu_u + \tilde{u}_i \times \sigma_u$. The geometry of this model is guaranteed to be well-behaved, irrespective of the amount of information collected for every term in the model. A similar approach is used for v_i. These transformations are treated as deterministic sites in numpyro.

3.2 Baseline Model – Equation (1)

Proposed functional form for the client:

$$f_i(t) = u_i e^{-v_i t/100}, \qquad y_i(t) = f_i(t) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_y^2) \tag{1}$$

Here $u_i > 0$ is the intercept at $t = 0$ representing the initial integrity, and $v_i > 0$ is the decay rate per unit of time. The divisor 100 is a numerical convenience kept throughout the analysis; v_i has a directly interpretable physical meaning (an example: $v_i = 5$ means the component will lose half of its integrity within 14 days

```
def baseline_model(comp_idx, days, y_obs=None):
    mu_u    = numpyro.sample('mu_u',    dist.Normal(90., 10.))
    sigma_u = numpyro.sample('sigma_u', dist.HalfNormal(10.))
    mu_v    = numpyro.sample('mu_v',    dist.Normal(5., 3.))
    sigma_v = numpyro.sample('sigma_v', dist.HalfNormal(3.))
    sigma_y = numpyro.sample('sigma_y', dist.HalfNormal(10.))
    with numpyro.plate('components', N_comp):
        u_raw = numpyro.sample('u_raw', dist.Normal(0., 1.))
        v_raw = numpyro.sample('v_raw', dist.Normal(0., 1.))
    u_i = numpyro.deterministic('u_i', mu_u + u_raw * sigma_u)
    v_i = numpyro.deterministic('v_i', mu_v + v_raw * sigma_v)
    f = u_i[comp_idx] * jnp.exp(-v_i[comp_idx] * days / 100.)
    numpyro.sample('y', dist.Normal(f, sigma_y), obs=y_obs)
```

Listing 1: Baseline hierarchical model (numpyro)

The prior form μ_u is set to $\mathcal{N}(90, 10^2)$match the client's recommendations regarding starting components close to a 100% intact state. Prior to μ_v follows $\mathcal{N}(5, 3^2)$ distribution, accounting for the client's range recommendation $v_i \in [1, 10]$. Half-normal distributions are used as priors for all variance parameters in the model, imposing positive constraints and still being weakly

informative priors. Observation noise σ_y is distributed as Half Normal(10) and is estimated along with other structural parameters of the model, rather than being fixed to any particular number. This allows the model to estimate the actual observation noise level rather than introducing possibly misspecified one. All five population-level parameters (μ_u, σ_u, μ_v, σ_v, σ_y) are globally shared by all 75 components and are fitted to the full dataset, thus allowing to exploit all available information on well-observed training components to infer about the sparse test set.

3.3 Enhanced Model - Equation (2)

The enhanced model accounts for the five physical attributes as shared predictors of the effective decay rate (2):

$$f_i(t) = u_i \exp\left(-\frac{\left(v_i + \sum_{j=1}^{5} w_j x_{ji} \right) t}{100} \right) \tag{2}$$

The weights w_j are shared among all the components: every weight represents the systematic effect of its corresponding physical attribute on the degradation rate for all the aircraft in the fleet. This is the major change from the baseline approach, where all the component-level variability in degradation rate is considered as random variability captured by σ_v.

A shrinkage prior is applied to the weights: $w_j \sim \mathcal{N}(0, \sigma_w^2)$ with $\sigma_w \sim$ HalfNormal(2). Such *regularisation* prevents the model from assigning high weight values to predictors unless there is enough evidence in the training data. This is very useful to avoid overfitting on small datasets. The prior on μ_v is changed to $\mathcal{N}(2, 2^2)$ since when taking into account the information in the predictors X, the variability of the baseline term v_i should be lower than in the baseline model. The major enhancements in the enhanced model code include:

```
def enhanced_model(comp_idx, days, X, y_obs=None):
 mu_u    = numpyro.sample('mu_u',    dist.Normal(90., 10.))
 sigma_u = numpyro.sample('sigma_u', dist.HalfNormal(10.))
 mu_v    = numpyro.sample('mu_v',    dist.Normal(2., 2.))
 sigma_v = numpyro.sample('sigma_v', dist.HalfNormal(2.))
 sigma_y = numpyro.sample('sigma_y', dist.HalfNormal(10.))
 sigma_w = numpyro.sample('sigma_w', dist.HalfNormal(2.))
 w       = numpyro.sample('w', dist.Normal(jnp.zeros(5),
                                           sigma_w * jnp.ones(5)))
 with numpyro.plate('components', N_comp):
     u_raw = numpyro.sample('u_raw', dist.Normal(0., 1.))
     v_raw = numpyro.sample('v_raw', dist.Normal(0., 1.))
 u_i = numpyro.deterministic('u_i', mu_u + u_raw * sigma_u)
 v_i = numpyro.deterministic('v_i', mu_v + v_raw * sigma_v)
 eff_v = v_i[comp_idx] + X[comp_idx] @ w
 f = u_i[comp_idx] * jnp.exp(-eff_v * days / 100.)
 numpyro.sample('y', dist.Normal(f, sigma_y), obs=y_obs)
```

Listing 2: Enhanced model additions (numpyro)

3.4 Sampler Configuration and Convergence Criteria

Each model was simulated using 2,000 warm-up and 2,000 sampling iterations with 4 parallel chains, which results in a total of 8,000 posterior samples. Calling numpyro.set_host_device_count(4) is critical since otherwise, all chains would run sequentially and no computation gain would be observed due to having multiple chains. Convergence was checked against three indicators from the specification: (i) no divergent transitions, (ii) $\hat{R} \leq 1.05$ for all parameters (with an exact

match to 1.00 being preferred), and (iii) an effective sample size n_{eff} larger than one tenth of the number of all samples (5). Divergent transitions mean that there are regions in the posterior that were not sampled well by the algorithm and are a clear indication of the model's misspecification or poor priors. The $\hat{R}$ indicator compares within-chain and between-chain variances and $\hat{R} = 1.00$ indicates convergence, whereas values deviating significantly from 1.00 show problems. n_{eff} measures the independence of MCMC draws in the correlated sample; a small value n_{eff} is indicative that more samples are required or that the parameterization should be changed. Table 1 demonstrates that all convergence requirements are satisfied for both models.

Model	**Divergences**	**Max $\hat{R}$**	**Min n_{eff}**	**Chains**
Baseline	0	1.00	1353	4
Enhanced	0	1.00	2607	4

Table 1: Sampler convergence diagnostics for both models. All parameters meet the $\hat{R} \leq 1.05$ criterion (5).

4 Evaluation

4.1 Baseline Model Results

The posterior statistics of the hyperparameters in the baseline model are summarized in Table 2. The components begin with about 90% integrity on average ($\mu_u = 89.70$std $= 0.88$) with an SD of $\sigma_u = 5.28\%$ for components' initial integrity level. This shows that the components do have some variation in terms of their initial condition; however, the variation is not significant. The mean value of the decay rate is $\mu_v = 3.92$ (std $= 0.21$). This implies that, given average conditions, the decay rate will reduce the component's integrity level by around 30% within the first 10 days. The variability in decay rates is $\sigma_v = 1.58$.

Parameter	**Mean**	**Std**	**5%**	**95%**	$\boldsymbol{n}_{\text{eff}}$	$\hat{\boldsymbol{R}}$
μ_u	89.70	0.88	88.27	91.15	5737	1.00
σ_u	5.28	0.81	3.96	6.60	4105	1.00
μ_v	3.92	0.21	3.58	4.28	1353	1.00
σ_v	1.58	0.17	1.31	1.86	2716	1.00
σ_y	5.11	0.13	4.88	5.32	11230	1.00

Table 2: Baseline model – posterior summary for global hyperparameters.

Figure 2 and Figure 3 display the result of calling `arviz.plot_trace`, as specified in the assignment. The traces are all of the classic "fuzzy caterpillar" shape for properly converged chains with no apparent trends or divergences. The kernel density estimate plots for the marginal posteriors have single modes and approximate Gaussian shapes for the scalar hyperparameters. The superimposed component parameter traces for u_i and v_i demonstrate that all 75 component parameters have been sampled successfully.

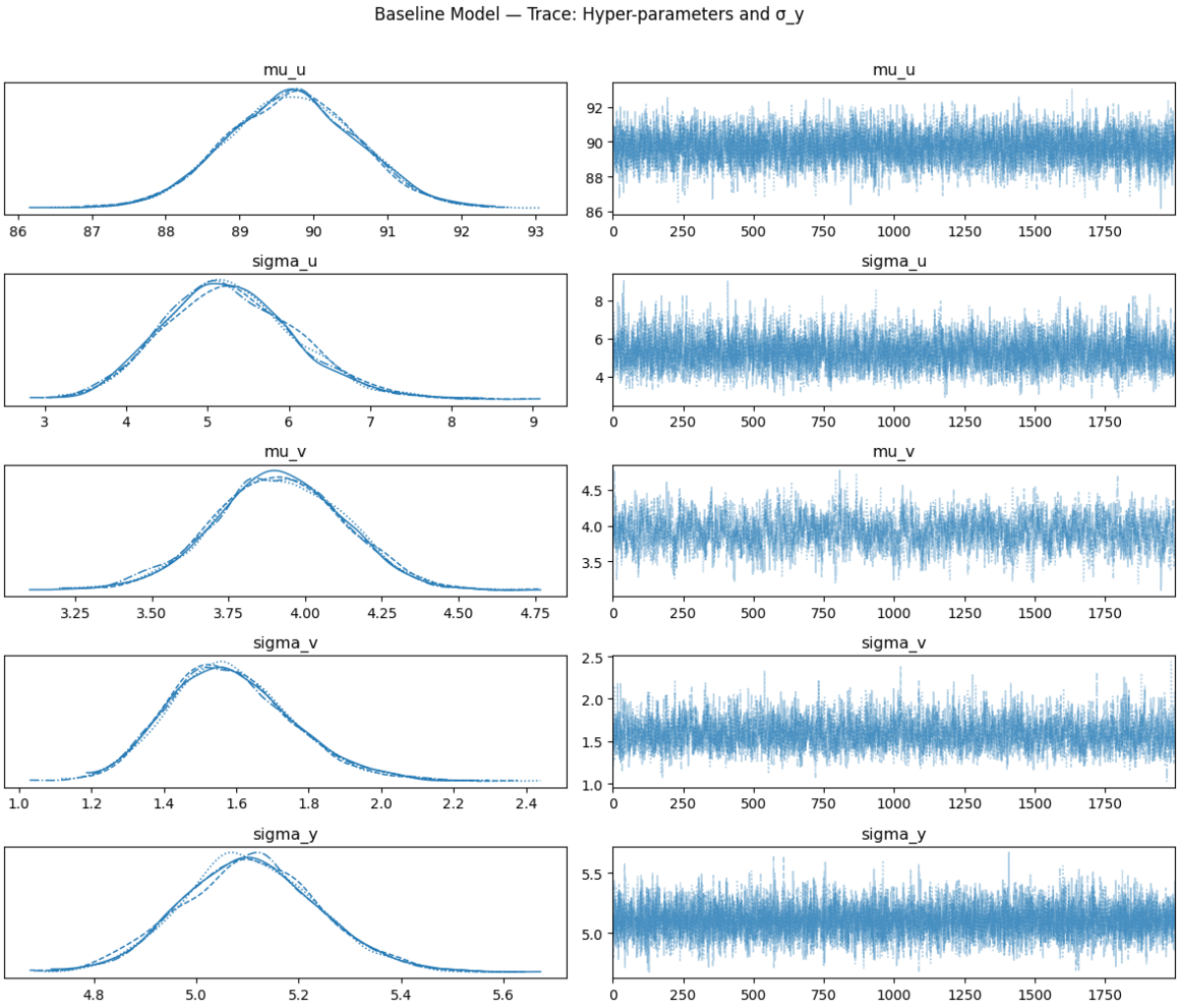

Figure 2: Baseline model – trace plots for hyperparameters μ_u, σ_u, μ_v, σ_v, σ_y. Left: marginal posterior KDEs. Right: traces over 2,000 iterations across 4 chains.

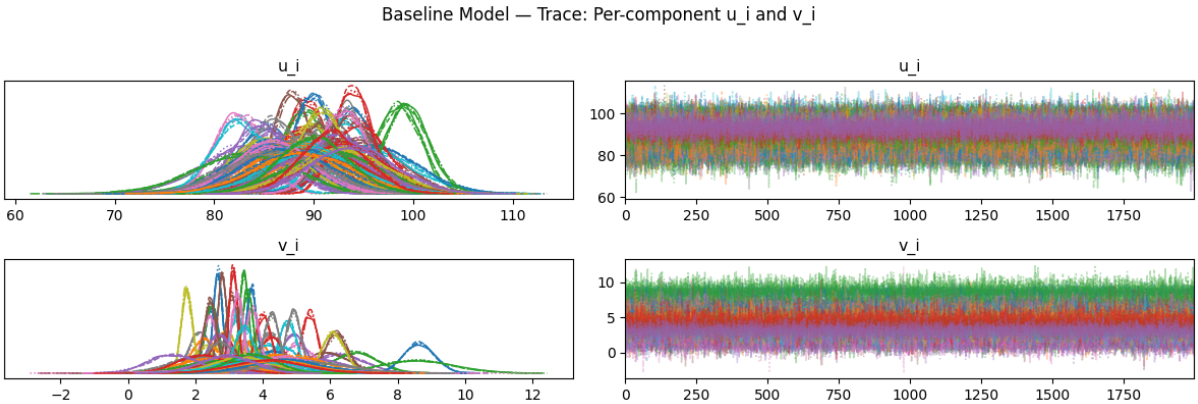

Figure 3: Baseline model – trace plots for per-component u_i and v_i (all 75 components overlaid).

In Figure 4, posterior predictive fits are presented for a well-sampled training component (K#0003, 40 samples) and three sparse test components. The posterior predictive mean function is illustrated in red; the blue band represents 95% of the function uncertainty ($\pm 1.96\,\sigma_f$); the orange band encompasses all predictive uncertainties, which include the measurement error

($\pm 1.96\sqrt{\sigma_f^2 + \sigma_y^2}$). As explained in Section 5.2.2 of the brief, the latter is calculated using the formula specified therein. It should be noted, in accordance with the hint provided in the problem statement, that the posterior mean function is calculated as the average of $f_i(t)$ obtained by sampling separately for each posterior sample of u_i and v_i.

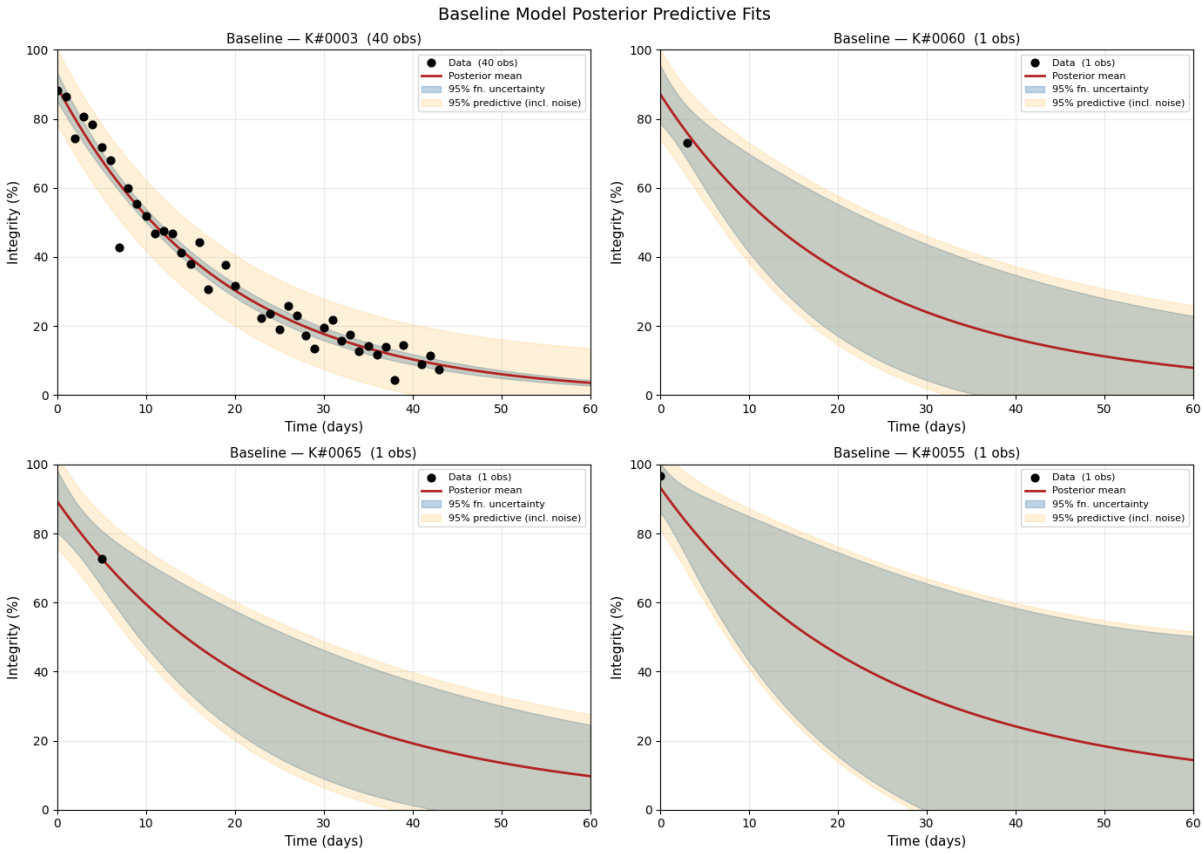

Figure 4: Baseline model – posterior predictive fits. Top-left: well-observed training component (40 observations). Remaining panels: three sparse test components. Blue band = 95% function uncertainty; orange band = 95% total predictive uncertainty including noise.

The well-monitored part fits perfectly and follows the data point closely. In the case of sparsely tested parts, the model correctly expands the uncertainty range, especially for the extrapolation range ($t > 10$), which reflects the amount of information available. The average prediction remains stable at a realistic value rather than becoming unrealistic, which is the direct result of partial pooling.

4.2 Enhanced Model Results

Table 3 illustrates the enhanced model's posterior estimates for global hyperparameters. The most significant differences from the baseline model are the decrease in μ_v from 3.92 to 1.99 and the drastic drop in σ_v from 1.58 to 0.34. These changes were anticipated and validate that the features X successfully explain a considerable portion of the variations in the decay rates observed by the baseline model. In summary, while the baseline simply accounts for the difference in decay rates among components, the enhanced model reveals the underlying reasons for such differences. The posterior estimate for the observation noise σ_y remains almost identical at 5.09%, indicating that the enhancements in the enhanced model resulted from improvements in structural modeling and not the absorption of observation noise in the weight parameter estimates. The posterior

for σ_w is estimated as 0.67 (95% CI $[0.29, 1.05]$), suggesting that the data supports a moderate level of variation in weight parameters.

Parameter	**Mean**	**Std**	**5%**	**95%**	$\boldsymbol{n}_{\text{eff}}$	$\hat{\boldsymbol{R}}$
μ_u	90.36	0.76	89.06	91.56	3159	1.00
σ_u	4.79	0.63	3.75	5.80	3117	1.00
μ_v	1.99	0.15	1.74	2.24	2776	1.00
σ_v	0.34	0.06	0.23	0.44	2607	1.00
σ_w	0.67	0.30	0.29	1.05	5360	1.00
σ_y	5.09	0.13	4.88	5.30	7724	1.00

Table 3: Enhanced model – posterior summary for global hyperparameters (feature weights shown separately in Table 4).

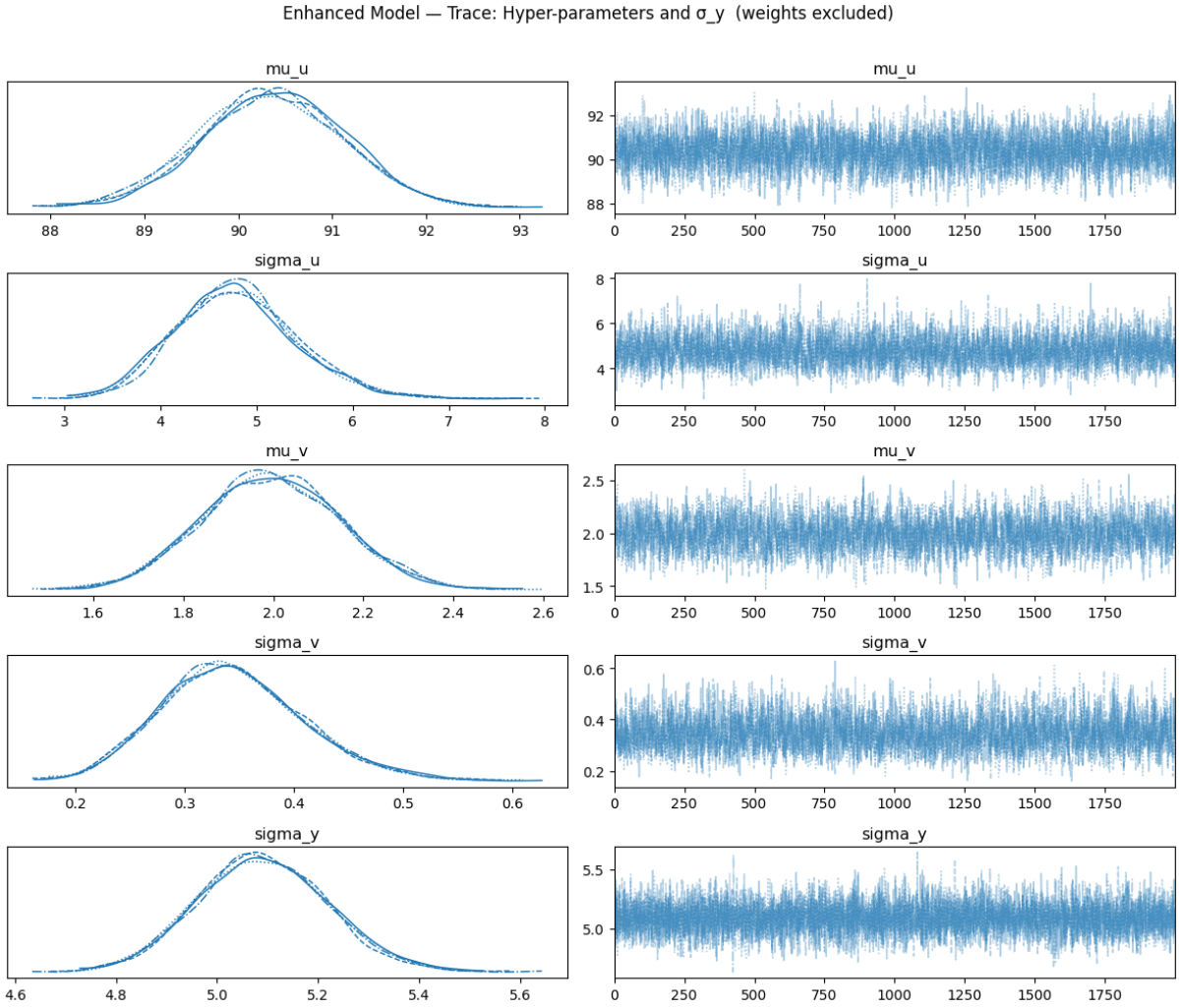

Figure 5: Enhanced model – trace plots for hyperparameters and σ_y (feature weights excluded; shown separately in Figure 9).

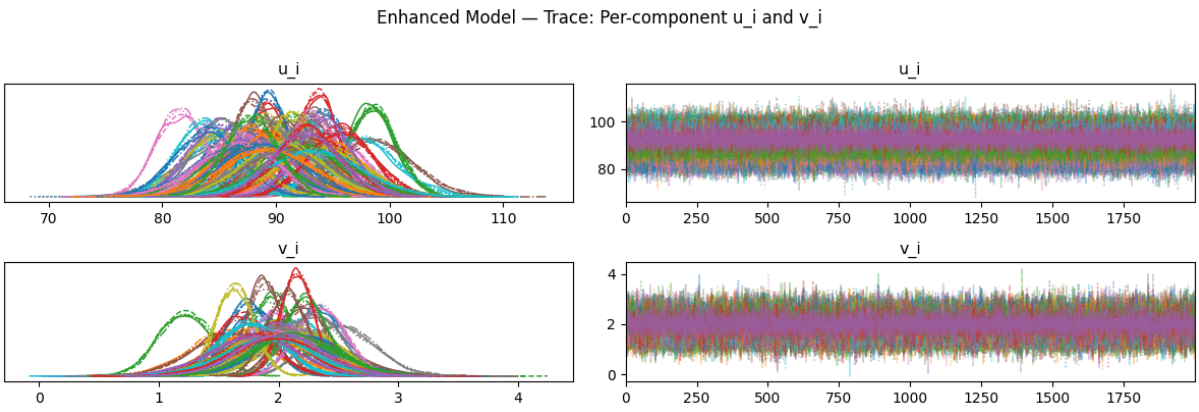

Figure 6: Enhanced model – trace plots for per-component u_i and v_i.

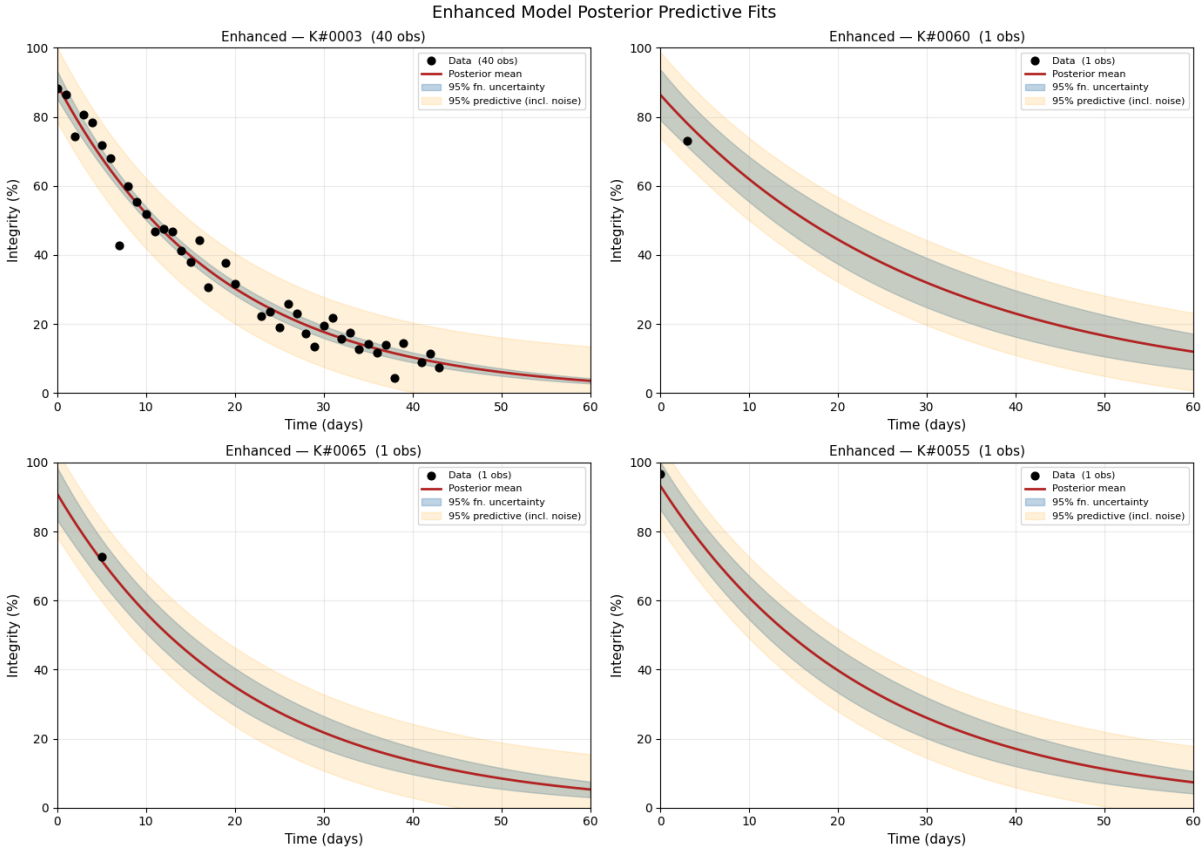

Figure 7: Enhanced model – posterior predictive fits for the same four components as Figure 4.

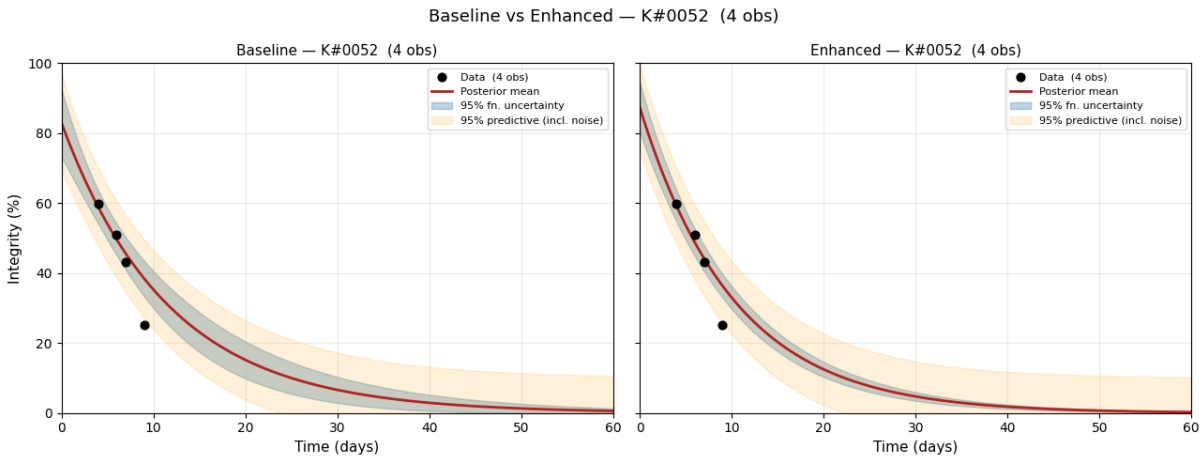

Figure 8: Side-by-side comparison of baseline (left) and enhanced (right) models on sparse test component K#0052 (4 observations). The enhanced model produces a tighter posterior by leveraging the component's high X3 value to inform the decay rate estimate.

4.3 Per-Feature Analysis – Potential Causes of Efficiency Loss

Weight posterior trace plots and violin plots can be seen in Figures 9 and 10. The posterior chains of all weights are well mixed with $\hat{R} = 1.00$, thus proving robust estimation of the impact of all five features. The difference between the X3 chain and the rest is easily noticeable, with X3 taking values in the interval around $+1.07$, while the other weights are near zero.

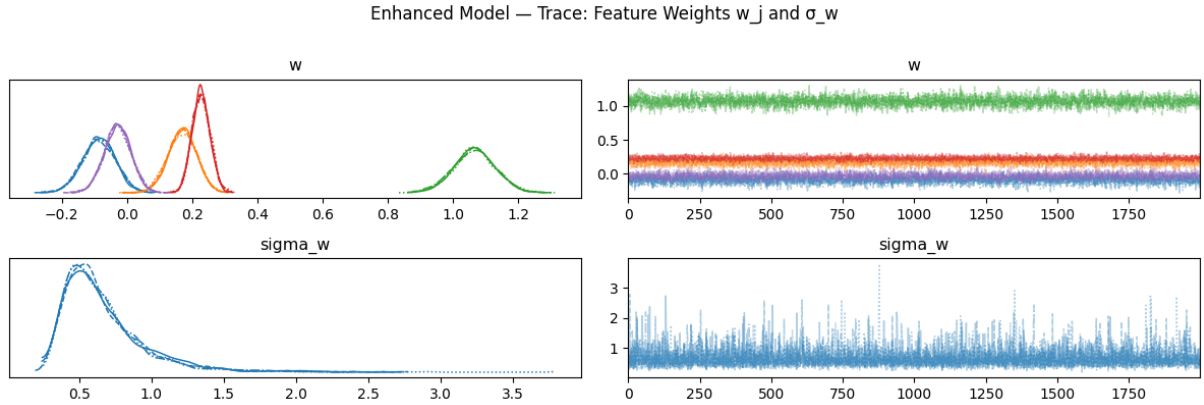

Figure 9: Enhanced model – trace plots for feature weights w_j and the weight prior scale σ_w. The w[X3] chain occupies a distinctly higher range than all others, confirming its dominant role.

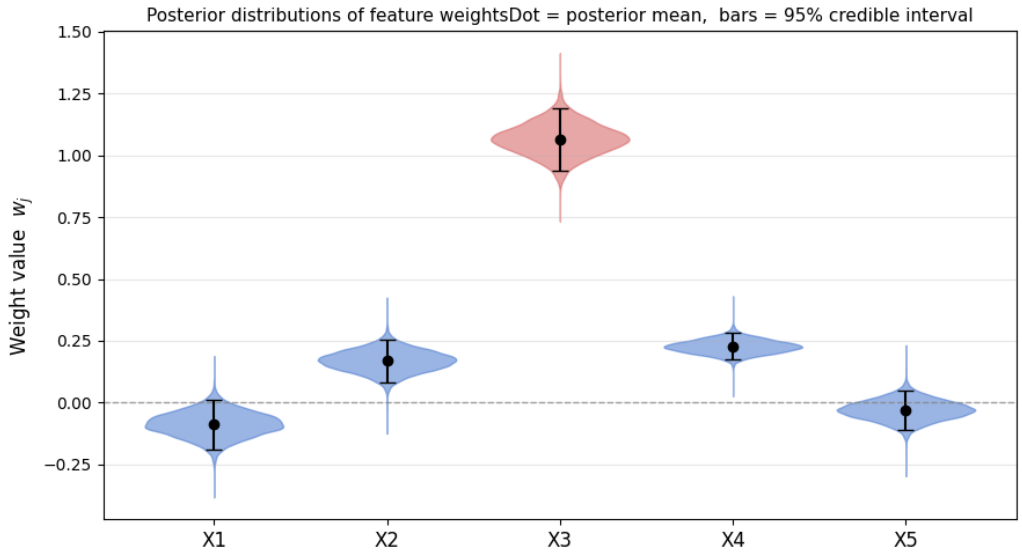

Figure 10: Posterior distributions of feature weights w_j. Dot = posterior mean; bars = 95% credible interval. X3 dominates; X1 and X5 are indistinguishable from zero.

Feature	**Mean**	**Std**	**2.5%**	**97.5%**	**Credibly $\neq 0$?**
X1	−0.087	0.051	−0.188	+0.011	No
X2	+0.169	0.044	+0.082	+0.253	**Yes**
X3	**+1.066**	**0.064**	+0.938	+1.193	**Yes**
X4	+0.227	0.027	+0.175	+0.281	**Yes**
X5	−0.029	0.041	−0.110	+0.050	No

Table 4: Posterior summary for feature weights w_j. A feature is classified as credibly non-zero if its 95% credible interval excludes zero entirely.

X3 is clearly the strongest determinant of the rate of degradation, having a posterior mean of +1.07, which is more than 16 posterior standard deviations away from zero. This constitutes a huge effect: one unit of X3 increases the decay rate by around 1.07 units. Since the values for X3 range between 0.08 and 6.82 in the dataset under consideration, we can conclude that a component on the higher end of this range receives an additional amount of 6.74 in decay on top of the base rate of v_i. Effectively, it will decay to only 30% integrity much faster than other components.

In contrast, X4 demonstrates a weakly significant positive effect with the posterior mean of +0.23 and the 95% credible interval ranging from +0.175 to +0.281. The same can be said about X2 which also gives a weakly significant positive effect with the posterior mean of +0.17 and the 95% credible interval from +0.08 to +0.25. Both X1 and X5 are non-significant; their 95% credible intervals are symmetrically located around zero, i.e., they do not give any reliable information about an effect being positive or negative.

The practical implication of this result seems straightforward. If variable X3 is related to some physically meaningful characteristic of components that is either easily controllable (load, temperature, hardness, etc.) or easy to measure before installation, then it can become the major screening characteristic. The components characterized by large values of X3 need to be regularly checked for wear-out. The authors strongly encourage the engineering team to investigate the physical meaning of X3. In case it is measurable before assembly and installation, it can become the basis of the risk-stratification approach, i.e., to introduce the special higher inspection

frequency regime for high-X3 components starting from the very beginning of their exploitation period, without waiting until their degradation starts to happen.

4.4 Blind Test Predictions

For each of the 25 items (indices 50-74), the probability $P(\text{integrity} \leq 30\% \text{at } t = 30)$ was calculated using the refined model. The probability value was computed by calculating the proportion of values that were ≤ 30 $f_i(30)$ in the 8,000 posterior samples. This Monte Carlo's counting methodology is similar to the integration of the entire posterior predictive distribution without any assumption of normality form (3). The full output is submitted in `predictions.csv`; Table 5 summarizes

ID	Index	*P*	ID	Index	*P*
K#0050	50	0.108	K#0063	63	0.001
K#0051	51	1.000	K#0064	64	0.000
K#0052	52	1.000	K#0065	65	0.998
K#0053	53	0.422	K#0066	66	1.000
K#0054	54	0.002	K#0067	67	1.000
K#0055	55	0.904	K#0068	68	0.023
K#0056	56	1.000	K#0069	69	1.000
K#0057	57	0.000	K#0070	70	0.998
K#0058	58	1.000	K#0071	71	0.936
K#0059	59	0.614	K#0072	72	0.048
K#0060	60	0.299	K#0073	73	0.981
K#0061	61	0.001	K#0074	74	0.000
K#0062	62	0.027			

Table 5: Blind test predictions – $P(\text{integrity} \leq 30\% \text{ at } t = 30)$ for test components 50–74.

The large range of probabilities (0.000 – 1.000) is indicative of the real variation in degradation rates between different components, which confirms the very strong influence of predictor X3. Components like K#0051, K#0052, K#0058, K#0066 and K#0067 can be expected to go under the 30% limit within 30 days, and all these components are characterised by large values of X3 in the training data set. On the other hand, components like K#0057, K#0064 and K#0074 have low values of X3 and slow decomposition; hence, they will stay over the 30% limit by the end of 30 days. Components with mid-level probability values (e.g., K#0053 with a probability of 0.42 and K#0059 with a probability of 0.61) can also be considered valuable in terms of monitoring since their probability values show that further sampling is needed to get more information about their behaviour. However, the quality of these probability values depends on how well the posterior samples are calibrated. The convergence statistics shown in Table 1 indicate perfect convergence ($\hat{R} = 1.0$ with no divergences). Therefore, the posterior samples accurately represent the actual posterior distribution, which means that the obtained probability values can be used for decision making.

5 Conclusion

Neither of the hierarchical Bayesian models shows any divergent transitions and gives $\hat{R} = 1.00$. The better model performs significantly better. In terms of corrosion, X3 is the leading predictor ($w_3 = 1.07 \pm 0.06$), followed by X4 and X2, while X1 and X5 barely does anything. As can be seen from above, the application of hierarchical Bayesian models was the right choice: due to partial pooling, it is now feasible to construct predictions from just one observation, and the uncertainty of such predictions is known in advance, which cannot be said about machine learning.

References

[1] Betancourt, M., 2017. A Conceptual Introduction to Hamiltonian Monte Carlo [Online]. arXiv.org. Available from: *https://arxiv.org/abs/1701.02434*.

[2] Bishop, C.M., 2006. Pattern Recognition and Machine Learning. Springer.

[3] Gelman, A., Carlin, J.B., Stern, H.S., Dunson, D.B., Vehtari, A. and Rubin, D.B., 2013. Bayesian Data Analysis. Boca Raton, Fl: Chapman Hall/CRC.

[4] Phan, D., Pradhan, N. and Jankowiak, M., 2019. Composable Effects for Flexible and Accelerated Probabilistic Programming in NumPyro [Online]. arXiv.org. Available from: *https://arxiv.org/abs/1912.11554*.

[5] Stan Docs, 2024. Stan User's Guide – Stan Docs [Online]. Available from: https://mc-stan.org/docs/stan-users-guide/.